Week 2 Submission

Peer-to-Peer Lending Matchmaking System

Regd No	25091A32G9, 25091A32K4
Problem Statement No	105
Title	Peer-to-Peer Lending Matchmaking System
Technology	Python, Flask, SQLite, NetworkX, Pandas/NumPy, ReportLab, OpenPyXL, Chart.js, Bootstrap, unittest/pytest
Data	Academic simulation using simulated test data - no real-money transactions

Where to run the code (read this first)

- **Where:** on your own laptop. Install **Python 3.10 or newer** from python.org (tick *Add Python to PATH*) and **VS Code**. Everything below is typed in the **VS Code terminal** (menu Terminal > New Terminal, or Ctrl+`).
- **Which folder:** open your **p2p_lending** folder (the one that contains **app.py**) with File > Open Folder. Every command must be run from this folder, not from a sub-folder, not in IDLE.
- **One-time setup:** run `python -m venv venv`, then activate it - Windows: `venv\Scripts\activate` (if PowerShell blocks it use Command Prompt or run `Set-ExecutionPolicy -Scope Process Bypass`), macOS/Linux: `source venv/bin/activate`. Then `pip install -r requirements.txt`.
- **Adding the files of a week:** in VS Code right-click the folder > New File, type the exact path shown in the heading (for example `routes/profiles.py`), paste the code. **[NEW]** = create the file, **[REPLACE]** = overwrite the old file completely, **[APPEND]** = paste at the very end of the existing file.
- **Web pages:** run `python app.py`, then open `http://127.0.0.1:5000` in your browser (stop the server with Ctrl+C). The pages load Bootstrap/Chart.js from the internet, so stay online.
- **Keep the weeks cumulative:** Week N needs the files of Weeks 1 to N-1. Your Week 1 zip is the starting point and none of its files is deleted later (only app.py and templates/index.html are replaced in Week 2, and config.py gets lines appended in Week 3).
- **Demo scripts** (`demo_weekN.py`) create their own throw-away databases in the data folder, so they never disturb your real database. To see the same demo data in the web pages: stop the server, delete `data/p2p_lending.db`, run `python demo_data.py`, start the server again.

Week 2: OOP Models and Profiles

Create the classes User, Lender, Borrower, LoanRequest, Match, Loan, Repayment and MatchingEngine, and build lender and borrower registration, loan-request form, profile validation and profile-status tracking. **Working output: profile-management module.**

Assignment steps and where they are done

Assignment step	Implemented in
Create classes User, Lender, Borrower, LoanRequest, Match, Loan, Repayment, MatchingEngine	models.py (7 classes, Lender and Borrower inherit from User) and matching_engine.py (MatchingEngine skeleton)
1. Lender registration and profile form	routes/profiles.py: /lenders/register
2. Borrower registration and loan-request form	routes/profiles.py: /borrowers/register and /requests/new
3. Profile validation	models.validate() + profile_service.py (required fields, number formats, ranges, duplicate e-mail, duplicate request)
4. Risk-appetite selection	drop-down low / medium / high in the lender form
5. Loan amount and term preferences	min/max loan amount and min/max duration in the lender form; amount and term in the request form
6. Profile-status tracking	pending -> active / rejected, active <-> inactive; buttons on /lenders and /borrowers; every change goes to audit_logs

Files of this week

File	Action	Purpose
app.py	REPLACE	Same routes as Week 1 + automatic registration of the blueprints of later weeks + menu
templates/index.html	REPLACE	Home page now uses the common layout
templates/base.html, form.html, table.html	NEW	Common layout, generic form page, generic table page
models.py	NEW	OOP classes with validation and behaviour
audit.py	NEW	log() writes one audit row
utils.py	NEW	db_conn() opens/closes a connection per request
profile_service.py	NEW	register_lender/borrower, create_loan_request, set_profile_status, queries
matching_engine.py	NEW	MatchingEngine skeleton (loads active lenders and pending requests)
routes/__init__.py, routes/profiles.py	NEW	Web pages for registration, lists and status buttons
demo_data.py, demo_week2.py	NEW	Demo dataset and console demo
tests/test_week2.py	NEW	14 tests

Class diagram

```
User (full_name, email, status)              LoanRequest (borrower_id, amount, term_months, status)
  |-- Lender   (risk_appetite, available_amount, loan range, duration range, return range)
  |-- Borrower (monthly_income, existing_obligations, credit_score, credit_history_years)
Match (lender_id, request_id, score components, allocated_amount, status, explanation)
Loan (principal, interest_rate, duration_months, outstanding_balance)   Repayment (due_date, amount_due, status)
MatchingEngine (loads active lenders and pending requests - full engine in Week 4)
```

How to run and check this week

1. `python tests/test_week2.py` - 14 tests, all ok

2. `python demo_week2.py` - console demo: profiles, validation errors, status tracking, audit trail

3. `python app.py` - open /lenders/register , /borrowers/register , /lenders , /borrowers (Approve button), /requests/new

Expected output of demo_week2.py

```
================================================================
1. Registered lenders
================================================================
#1 Conservative Carla   appetite=low    available=   300,000 loans 10,000-150,000  6-36m  return 7-11%  [active]
#2 Balanced Bala        appetite=medium available=   250,000 loans 5,000-200,000  6-48m  return 9-15%  [active]
#3 Aggressive Arjun     appetite=high   available=   250,000 loans 5,000-300,000  3-60m  return 12-22%  [active]
#4 Small Sita           appetite=medium available=    20,000 loans 2,000-30,000  3-24m  return 9-16%  [active]
#5 Cohort Chitra        appetite=high   available=   100,000 loans 50,000-300,000  12-60m  return 12-22%  [active]

================================================================
2. Registered borrowers
================================================================
#1 Ravi Kumar     income=  60,000 obligations=  8,000 credit=780 history=6.0y  DTI=13%  [active]
#2 Meena Iyer     income=  25,000 obligations=  9,000 credit=540 history=1.5y  DTI=36%  [active]
#3 Sunil Verma    income=  45,000 obligations= 12,000 credit=650 history=3.0y  DTI=27%  [active]
#4 Anita Das      income=  30,000 obligations= 18,000 credit=600 history=2.0y  DTI=60%  [active]
#5 Kiran Rao      income=  90,000 obligations=  5,000 credit=820 history=10.0y  DTI=6%  [active]
#6 Pooja Nair     income=  35,000 obligations=  6,000 credit=700 history=4.0y  DTI=17%  [active]

================================================================
3. Loan requests
================================================================
#1 Ravi Kumar     amount=   50,000 term=12m purpose=Education        [pending]
#2 Meena Iyer     amount=   20,000 term=6m purpose=Medical          [pending]
#3 Sunil Verma    amount=   80,000 term=24m purpose=Home renovation  [pending]
#4 Anita Das      amount=   40,000 term=12m purpose=Business         [pending]
#5 Kiran Rao      amount=   15,000 term=12m purpose=Vehicle repair   [pending]
#6 Kiran Rao      amount=  120,000 term=36m purpose=Home loan top-up [pending]
#7 Pooja Nair     amount=  280,000 term=48m purpose=Start-up capital [pending]
================================================================
4. Validation rejects wrong data
================================================================
- missing name + invalid email: rejected -> Full name must have at least 2 characters.
- unknown risk appetite: rejected -> Risk appetite must be one of ('low', 'medium', 'high').
- min duration above max: rejected -> Duration preference must satisfy 3 <= minimum <= maximum <= 60 months.
- text in a number field: rejected -> Available amount must be a valid number.
- duplicate email: rejected -> This email address is already registered.
- duplicate loan request: rejected -> Duplicate request: request #1 with the same amount and term is already open.

================================================================
5. Profile-status tracking (pending -> active -> inactive)
================================================================
registered lender status : pending
after approval           : active
after deactivation       : inactive
illegal move inactive->rejected blocked: Cannot change lender profile from 'inactive' to 'rejected'.

================================================================
6. Audit trail (last 5 entries)
================================================================
('lender', 6, 'PROFILE_STATUS_CHANGE', 'active', 'inactive')
('lender', 6, 'PROFILE_STATUS_CHANGE', 'pending', 'active')
('lender', 6, 'REGISTERED', None, 'pending')
('loan_request', 7, 'CREATED', None, 'pending')
('loan_request', 6, 'CREATED', None, 'pending')
```

[REPLACE] *app.py*

```
"""Flask application entry point.

Week 1 created the skeleton.  Week 2 adds automatic blueprint registration, so
the routes written in Weeks 2-6 are picked up when their files exist and this
file never needs to change again.
"""
import importlib

from flask import Flask, jsonify, render_template

import config
from database import get_connection, init_db, list_tables
from utils import db_conn

BLUEPRINT_MODULES = [
    "routes.profiles",    # Week 2
    "routes.scoring",     # Week 3
    "routes.matching",    # Week 4
    "routes.loans",       # Week 5
    "routes.dashboard",   # Week 6
    "routes.reports",     # Week 6
    "routes.api",         # Week 6
]

# (menu label, endpoint) - a menu item appears only when its endpoint exists
NAV_ITEMS = [
    ("Register Lender", "profiles.register_lender"),
    ("Register Borrower", "profiles.register_borrower"),
    ("Lenders", "profiles.lenders"),
    ("Borrowers", "profiles.borrowers"),
    ("Requests", "profiles.requests"),
    ("Risk Scores", "scoring.risk_scores"),
    ("Scored Pairs", "scoring.pairs"),
    ("Recommendations", "matching.recommendations"),
    ("Pending Funding", "loans.pending"),
    ("Loans", "loans.loans"),
    ("Admin", "dashboard.admin"),
    ("Audit", "reports.audit_log"),
]
```

```
def create_app(db_path=None):
    app = Flask(__name__)
    app.config["SECRET_KEY"] = config.SECRET_KEY
    if db_path:
        config.DATABASE_PATH = db_path
    init_db()

    for module_name in BLUEPRINT_MODULES:
        try:
            module = importlib.import_module(module_name)
        except ModuleNotFoundError as exc:
            if exc.name != module_name:          # a genuine error inside an existing module
                raise
            continue                              # this week's file does not exist yet
        app.register_blueprint(module.bp)

    @app.context_processor
    def inject_navigation():
        return {"nav_items": [(label, ep) for label, ep in NAV_ITEMS if ep in app.view_functions]}

    @app.route("/")
    def home():
        counts = {}
        with db_conn() as conn:
            for table in list_tables():
                counts[table] = conn.execute(f"SELECT COUNT(*) FROM {table}").fetchone()[0]
        return render_template("index.html", counts=counts)

    @app.route("/health")
    def health():
        return jsonify(status="ok", project="P2P Lending Matchmaking System")

    @app.route("/api/tables")
    def api_tables():
        conn = get_connection()
        try:
            return jsonify({t: conn.execute(f"SELECT COUNT(*) FROM {t}").fetchone()[0] for t in list_tables()})
        finally:
            conn.close()

    @app.route("/api/rules")
    def api_rules():
        return jsonify(
            rate_by_risk=config.RATE_BY_RISK,
            score_weights=config.SCORE_WEIGHTS,
            max_debt_to_income=config.MAX_DEBT_TO_INCOME,
            loan_amount_limits=[config.MIN_LOAN_AMOUNT, config.MAX_LOAN_AMOUNT],
            term_limits_months=[config.MIN_TERM_MONTHS, config.MAX_TERM_MONTHS],
        )

    return app


if __name__ == "__main__":
    create_app().run(debug=True)
```

[REPLACE] *templates/index.html*

```
{% extends "base.html" %}
{% block content %}
<div class="p-4 mb-4 bg-white rounded shadow-sm">
  <h1 class="h3">P2P Lending Matchmaking System</h1>
  <p class="text-muted mb-0">Problem Statement 105 &mdash; graph-based risk-compatibility matching of lenders
  and borrowers. Academic simulation, no real money.</p>
</div>
<div class="card shadow-sm">
  <div class="card-header">Database tables</div>
  <table class="table table-sm mb-0">
    <thead><tr><th>Table</th><th class="text-end">Rows</th></tr></thead>
    <tbody>
      {% for table, count in counts.items() %}
      <tr><td>{{ table }}</td><td class="text-end">{{ count }}</td></tr>
      {% endfor %}
    </tbody>
  </table>
</div>
{% endblock %}
```

[NEW] templates/base.html

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>{% block title %}P2P Lending Matchmaking System{% endblock %}</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body class="bg-light">
<nav class="navbar navbar-dark bg-primary mb-4">
  <div class="container">
    <a class="navbar-brand" href="{{ url_for('home') }}">P2P Lending Matchmaker</a>
    <ul class="navbar-nav flex-row flex-wrap gap-3">
      {% for label, endpoint in nav_items %}
      <li class="nav-item"><a class="nav-link py-0" href="{{ url_for(endpoint) }}">{{ label }}</a></li>
      {% endfor %}
    </ul>
  </div>
</nav>
<main class="container pb-5">
  {% with messages = get_flashed_messages(with_categories=true) %}
    {% for category, message in messages %}
      <div class="alert alert-{{ category }} alert-dismissible fade show" role="alert">
        {{ message }}<button type="button" class="btn-close" data-bs-dismiss="alert"></button>
      </div>
    {% endfor %}
  {% endwith %}
  {% block content %}{% endblock %}
</main>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
{% block scripts %}{% endblock %}
</body>
</html>
```

[NEW] templates/form.html

```
{% extends "base.html" %}
{% block content %}
<div class="card shadow-sm" style="max-width: 640px;">
  <div class="card-body">
    <h2 class="h4 mb-3">{{ title }}</h2>
    <form method="post" novalidate>
      {% for f in fields %}
      <div class="mb-3">
        <label class="form-label" for="{{ f.name }}">{{ f.label }}</label>
        {% if f.type == 'select' %}
        <select class="form-select" id="{{ f.name }}" name="{{ f.name }}">
          {% for value, label in f.options %}
          <option value="{{ value }}" {% if values.get(f.name, '') | string == value | string %}selected{% endif %}>{{ label }}</option>
          {% endfor %}
        </select>
        {% else %}
        <input class="form-control" id="{{ f.name }}" name="{{ f.name }}" type="{{ f.type or 'text' }}"
               {% if f.step %}step="{{ f.step }}"{% endif %}
               value="{% if f.type != 'password' %}{{ values.get(f.name, '') }}{% endif %}">
        {% endif %}
      </div>
      {% endfor %}
      <button class="btn btn-primary" type="submit">{{ submit_label }}</button>
    </form>
  </div>
</div>
{% endblock %}
```

[NEW] templates/table.html

```
{% extends "base.html" %}
{% macro action_button(a) %}
  {% if a.method == 'post' %}
  <form method="post" action="{{ a.url }}" class="d-inline">
    <button class="btn btn-sm btn-{{ a.style or 'primary' }}" type="submit">{{ a.label }}</button>
  </form>
  {% else %}
  <a class="btn btn-sm btn-{{ a.style or 'primary' }}" href="{{ a.url }}">{{ a.label }}</a>
  {% endif %}
{% endmacro %}
{% block content %}
<div class="d-flex justify-content-between align-items-center mb-3">
  <h2 class="h4 mb-0">{{ title }}</h2>
  <div class="d-flex gap-2">{% for a in top_actions or [] %}{{ action_button(a) }}{% endfor %}</div>
</div>
{% if intro %}<p class="text-muted">{{ intro }}</p>{% endif %}
<div class="card shadow-sm">
  <div class="table-responsive">
    <table class="table table-sm table-hover align-middle mb-0">
      <thead class="table-light">
        <tr>{% for key, label in columns %}<th>{{ label }}</th>{% endfor %}<th></th></tr>
      </thead>
      <tbody>
        {% for row in rows %}
        <tr>
          {% for key, label in columns %}
          <td>{% if row.get(key) is none %}{% else %}{{ row.get(key) }}{% endif %}</td>
          {% endfor %}
          <td class="text-end text-nowrap">{% for a in row.get('_actions', []) %}{{ action_button(a) }} {% endfor %}</td>
        </tr>
        {% else %}
        <tr><td colspan="{{ columns|length + 1 }}" class="text-center text-muted py-4">No records yet.</td></tr>
        {% endfor %}
      </tbody>
    </table>
  </div>
</div>
{% endblock %}
```

[NEW] models.py

```
"""Week 2 - OOP domain models: User, Lender, Borrower, LoanRequest, Match, Loan, Repayment.

Every class validates its own data (validate() returns a list of error strings)
and exposes small behaviours that the scoring / matching engines reuse.
"""
import math
import re
from dataclasses import asdict, dataclass

import config

EMAIL_RE = re.compile(r"^[^@\s]+@[^@\s]+\.[^@\s]+$")
MAX_EXPECTED_RETURN = 50.0          # annual %, sanity limit for lender profiles
PROFILE_STATUSES = ("pending", "active", "inactive", "rejected")
USER_STATUSES = ("active", "inactive", "suspended")


class ValidationError(Exception):
    """Raised by the services when data is missing or invalid."""

    def __init__(self, errors):
        if isinstance(errors, str):
            errors = [errors]
        self.errors = list(errors)
        super().__init__("; ".join(self.errors))


def _is_number(value):
    return isinstance(value, (int, float)) and not isinstance(value, bool) and math.isfinite(value)


def calculate_emi(principal, annual_rate_pct, months):
    """Equated monthly instalment (standard reducing-balance formula)."""
    if months <= 0:
        raise ValueError("months must be positive")
    r = annual_rate_pct / 12.0 / 100.0
    if r == 0:
        return round(principal / months, 2)
    return round(principal * r / (1 - (1 + r) ** (-months)), 2)


# ---------------------------------------------------------------------------
# Users
# ---------------------------------------------------------------------------
class User:
    """Base class for every person in the system."""

    ROLE = "admin"

    def __init__(self, full_name, email, status="active", user_id=None, password_hash=""):
        self.user_id = user_id
        self.full_name = (full_name or "").strip()
        self.email = (email or "").strip().lower()
        self.status = status
        self.password_hash = password_hash
    @property
```

```
    def role(self):
        return self.ROLE

    def validate(self):
        errors = []
        if len(self.full_name) < 2:
            errors.append("Full name must have at least 2 characters.")
        if not EMAIL_RE.match(self.email):
            errors.append("A valid email address is required.")
        if self.status not in USER_STATUSES:
            errors.append(f"User status must be one of {USER_STATUSES}.")
        return errors

    def is_active(self):
        return self.status == "active"

    def to_dict(self):
        data = dict(vars(self))
        data.pop("password_hash", None)
        data["role"] = self.role
        return data


class Lender(User):
    ROLE = "lender"

    def __init__(self, full_name, email, risk_appetite, available_amount,
                 min_loan_amount, max_loan_amount, min_duration_months, max_duration_months,
                 min_expected_return, max_expected_return, profile_status="pending",
                 lender_id=None, **user_kwargs):
        super().__init__(full_name, email, **user_kwargs)
        self.lender_id = lender_id
        self.risk_appetite = (risk_appetite or "").strip().lower() if isinstance(risk_appetite, str) else risk_appetite
        self.available_amount = available_amount
        self.min_loan_amount = min_loan_amount
        self.max_loan_amount = max_loan_amount
        self.min_duration_months = min_duration_months
        self.max_duration_months = max_duration_months
        self.min_expected_return = min_expected_return
        self.max_expected_return = max_expected_return
        self.profile_status = profile_status

    # ---- validation ----------------------------------------------------------
    def validate(self):
        errors = super().validate()
        if self.risk_appetite not in config.RISK_LEVELS:
            errors.append(f"Risk appetite must be one of {config.RISK_LEVELS}.")
        numeric = {
            "Available amount": self.available_amount, "Minimum loan amount": self.min_loan_amount,
            "Maximum loan amount": self.max_loan_amount, "Minimum duration": self.min_duration_months,
            "Maximum duration": self.max_duration_months, "Minimum expected return": self.min_expected_return,
            "Maximum expected return": self.max_expected_return}
        bad = [label for label, v in numeric.items() if not _is_number(v)]
        if bad:
            return errors + [f"{label} must be a number." for label in bad]
        if self.available_amount <= 0:
            errors.append("Available amount must be greater than 0.")
        if not config.MIN_LOAN_AMOUNT <= self.min_loan_amount <= self.max_loan_amount <= config.MAX_LOAN_AMOUNT:
            errors.append(f"Loan amount preference must satisfy {config.MIN_LOAN_AMOUNT} <= minimum <= "
                          f"maximum <= {config.MAX_LOAN_AMOUNT}.")
        if not config.MIN_TERM_MONTHS <= self.min_duration_months <= self.max_duration_months <= config.MAX_TERM_MONTHS:
            errors.append(f"Duration preference must satisfy {config.MIN_TERM_MONTHS} <= minimum <= "
                          f"maximum <= {config.MAX_TERM_MONTHS} months.")
        if not 0 <= self.min_expected_return <= self.max_expected_return <= MAX_EXPECTED_RETURN:
            errors.append(f"Expected return must satisfy 0 <= minimum <= maximum <= {MAX_EXPECTED_RETURN}%.")
        if self.profile_status not in PROFILE_STATUSES:
            errors.append(f"Profile status must be one of {PROFILE_STATUSES}.")
        return errors

    # ---- behaviour used by the matching engine -----------------------------
    @property
    def risk_level(self):
        return config.RISK_ORDER[self.risk_appetite]

    def is_matchable(self):
        return self.is_active() and self.profile_status == "active"

    def accepts_risk(self, category):
        return config.RISK_ORDER[category] <= self.risk_level

    def accepts_amount(self, amount):
        return self.min_loan_amount <= amount <= self.max_loan_amount

    def accepts_duration(self, months):
        return self.min_duration_months <= months <= self.max_duration_months

    def accepts_rate(self, rate):
        return self.min_expected_return <= rate <= self.max_expected_return

    def can_fund(self, amount):
        return self.available_amount + 1e-9 >= amount

    @classmethod
    def from_row(cls, row):
        return cls(row["full_name"], row["email"], row["risk_appetite"], row["available_amount"],
                   row["min_loan_amount"], row["max_loan_amount"], row["min_duration_months"],
                   row["max_duration_months"], row["min_expected_return"], row["max_expected_return"],
                   profile_status=row["profile_status"], lender_id=row["lender_id"],
                   user_id=row["user_id"], status=row["status"])
class Borrower(User):
```

```
    ROLE = "borrower"

    def __init__(self, full_name, email, monthly_income, existing_obligations, credit_score,
                 credit_history_years, profile_status="pending", borrower_id=None, **user_kwargs):
        super().__init__(full_name, email, **user_kwargs)
        self.borrower_id = borrower_id
        self.monthly_income = monthly_income
        self.existing_obligations = existing_obligations
        self.credit_score = credit_score
        self.credit_history_years = credit_history_years
        self.profile_status = profile_status

    def validate(self):
        errors = super().validate()
        numeric = {"Monthly income": self.monthly_income, "Existing obligations": self.existing_obligations,
                   "Credit score": self.credit_score, "Credit history": self.credit_history_years}
        bad = [label for label, v in numeric.items() if not _is_number(v)]
        if bad:
            return errors + [f"{label} must be a number." for label in bad]
        if self.monthly_income <= 0:
            errors.append("Monthly income must be greater than 0.")
        if self.existing_obligations < 0:
            errors.append("Existing obligations cannot be negative.")
        if not 300 <= self.credit_score <= 900:
            errors.append("Credit score must be between 300 and 900.")
        if not 0 <= self.credit_history_years <= 60:
            errors.append("Credit history must be between 0 and 60 years.")
        if self.profile_status not in PROFILE_STATUSES:
            errors.append(f"Profile status must be one of {PROFILE_STATUSES}.")
        return errors

    def debt_to_income(self):
        return self.existing_obligations / self.monthly_income

    def is_matchable(self):
        return self.is_active() and self.profile_status == "active"

    @classmethod
    def from_row(cls, row):
        return cls(row["full_name"], row["email"], row["monthly_income"], row["existing_obligations"],
                   row["credit_score"], row["credit_history_years"], profile_status=row["profile_status"],
                   borrower_id=row["borrower_id"], user_id=row["user_id"], status=row["status"])


# ---------------------------------------------------------------------------
# Lending objects
# ---------------------------------------------------------------------------
@dataclass
class LoanRequest:
    borrower_id: int
    amount: float
    term_months: int
    purpose: str = ""
    status: str = "pending"
    request_id: int = None

    def validate(self):
        errors = []
        if not _is_number(self.amount):
            errors.append("Loan amount must be a number.")
        elif not config.MIN_LOAN_AMOUNT <= self.amount <= config.MAX_LOAN_AMOUNT:
            errors.append(f"Loan amount must be between {config.MIN_LOAN_AMOUNT} and {config.MAX_LOAN_AMOUNT}.")
        if not _is_number(self.term_months) or int(self.term_months) != self.term_months:
            errors.append("Repayment term must be a whole number of months.")
        elif not config.MIN_TERM_MONTHS <= self.term_months <= config.MAX_TERM_MONTHS:
            errors.append(f"Repayment term must be between {config.MIN_TERM_MONTHS} and "
                          f"{config.MAX_TERM_MONTHS} months.")
        if len(self.purpose or "") > 200:
            errors.append("Purpose must be at most 200 characters.")
        return errors

    def monthly_instalment(self, annual_rate_pct):
        return calculate_emi(self.amount, annual_rate_pct, int(self.term_months))

    def to_dict(self):
        return asdict(self)

    @classmethod
    def from_row(cls, row):
        return cls(row["borrower_id"], row["amount"], row["term_months"], row["purpose"] or "",
                   row["status"], row["request_id"])


@dataclass
class Match:
    lender_id: int
    request_id: int
    compatibility_score: float = 0.0
    risk_fit: float = 0.0
    amount_fit: float = 0.0
    term_fit: float = 0.0
    return_fit: float = 0.0
    allocated_amount: float = 0.0
    status: str = "recommended"
    explanation: str = ""
    match_id: int = None

    def is_selected(self):
        return self.status in ("recommended", "accepted") and self.allocated_amount > 0
    def to_dict(self):
```

```
        return asdict(self)


@dataclass
class Loan:
    match_id: int
    lender_id: int
    borrower_id: int
    principal: float
    interest_rate: float
    duration_months: int
    outstanding_balance: float = None
    status: str = "active"
    start_date: str = None
    loan_id: int = None

    def monthly_instalment(self):
        return calculate_emi(self.principal, self.interest_rate, self.duration_months)

    def to_dict(self):
        return asdict(self)


@dataclass
class Repayment:
    loan_id: int
    installment_no: int
    due_date: str
    amount_due: float
    amount_paid: float = 0.0
    status: str = "pending"
    paid_date: str = None
    repayment_id: int = None

    def is_settled(self):
        return self.status in ("paid", "late")

    def to_dict(self):
        return asdict(self)
```

[NEW] audit.py

```
"""Audit trail helper - every important status change is written to audit_logs."""


def log(conn, entity_type, entity_id, action, old_value=None, new_value=None, actor="system", details=None):
    """Insert one audit row.  The caller is responsible for conn.commit().

    The audit_logs table (Week 1) has old_value / new_value columns, so any extra
    explanation (`details`) is stored after the new value, e.g. 'active | principal 5,000'.
    """
    if details:
        new_value = f"{new_value} | {details}" if new_value is not None else details
    conn.execute(
        "INSERT INTO audit_logs(entity_type, entity_id, action, old_value, new_value, actor)"
        " VALUES (?, ?, ?, ?, ?, ?)",
        (entity_type, entity_id, action,
         None if old_value is None else str(old_value),
         None if new_value is None else str(new_value), actor))
```

[NEW] utils.py

```
"""Small helpers shared by the Flask routes."""
from contextlib import contextmanager

from database import get_connection


@contextmanager
def db_conn():
    """Open a database connection for one request and always close it."""
    conn = get_connection()
    try:
        yield conn
    finally:
        conn.close()
```

[NEW] *profile_service.py*

```
"""Week 2 - profile management: registration, validation, status tracking.

All functions take an open sqlite3 connection (`conn`) so they work inside
Flask routes, scripts and tests.  Invalid input raises ValidationError.
"""
import math
import sqlite3

from werkzeug.security import generate_password_hash

import audit
from models import Borrower, Lender, LoanRequest, ValidationError

MIN_PASSWORD_LENGTH = 8

# allowed profile-status transitions (profile status tracking)
STATUS_TRANSITIONS = {
    "pending": {"active", "rejected"},
    "active": {"inactive"},
    "inactive": {"active"},
    "rejected": {"pending"},
}

_LENDER_SQL = ("SELECT l.*, u.full_name, u.email, u.status FROM lenders l "
               "JOIN users u ON u.user_id = l.user_id")
_BORROWER_SQL = ("SELECT b.*, u.full_name, u.email, u.status FROM borrowers b "
                 "JOIN users u ON u.user_id = b.user_id")


# ---------------------------------------------------------------------------
# input parsing helpers
# ---------------------------------------------------------------------------
def _text(data, key):
    value = data.get(key)
    return "" if value is None else str(value).strip()
def _number(data, key, label, cast, errors):
    raw = data.get(key)
    if raw is None or str(raw).strip() == "":
        errors.append(f"{label} is required.")
        return None
    try:
        value = cast(str(raw).strip().replace(",", ""))
    except ValueError:
        errors.append(f"{label} must be a valid {'whole number' if cast is int else 'number'}.")
        return None
    if isinstance(value, float) and not math.isfinite(value):
        errors.append(f"{label} must be a finite number.")
        return None
    return value


def _check_password(data, errors):
    password = str(data.get("password") or "")
    if len(password) < MIN_PASSWORD_LENGTH:
        errors.append(f"Password must have at least {MIN_PASSWORD_LENGTH} characters.")
        return None
    return generate_password_hash(password)


def _insert_user(conn, person, role, password_hash):
    cur = conn.execute(
        "INSERT INTO users(full_name, email, password_hash, role, status) VALUES (?, ?, ?, ?, 'active')",
        (person.full_name, person.email, password_hash, role))
    return cur.lastrowid


def _email_taken(conn, email):
    return conn.execute("SELECT 1 FROM users WHERE email = ?", (email,)).fetchone() is not None


# ---------------------------------------------------------------------------
# registration
# ---------------------------------------------------------------------------
def register_lender(conn, data, auto_activate=False):
    """Validate and store a lender.  Returns the new lender_id."""
    errors = []
    f = lambda key, label: _number(data, key, label, float, errors)       # noqa: E731
    i = lambda key, label: _number(data, key, label, int, errors)         # noqa: E731
    lender = Lender(
        _text(data, "full_name"), _text(data, "email"), _text(data, "risk_appetite"),
        f("available_amount", "Available amount"), f("min_loan_amount", "Minimum loan amount"),
        f("max_loan_amount", "Maximum loan amount"), i("min_duration_months", "Minimum duration"),
        i("max_duration_months", "Maximum duration"), f("min_expected_return", "Minimum expected return"),
        f("max_expected_return", "Maximum expected return"))
    password_hash = _check_password(data, errors)
    if errors:
        raise ValidationError(errors)
    errors = lender.validate()
    if lender.email and _email_taken(conn, lender.email):
        errors.append("This email address is already registered.")
    if errors:
        raise ValidationError(errors)

    status = "active" if auto_activate else "pending"
    try:
        user_id = _insert_user(conn, lender, "lender", password_hash)
        cur = conn.execute(
            "INSERT INTO lenders(user_id, risk_appetite, available_amount, min_loan_amount, max_loan_amount,"
```

```
            " min_duration_months, max_duration_months, min_expected_return, max_expected_return, profile_status)"
            " VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)",
            (user_id, lender.risk_appetite, lender.available_amount, lender.min_loan_amount,
             lender.max_loan_amount, lender.min_duration_months, lender.max_duration_months,
             lender.min_expected_return, lender.max_expected_return, status))
        lender_id = cur.lastrowid
        audit.log(conn, "lender", lender_id, "REGISTERED", None, status, actor=lender.email)
        conn.commit()
        return lender_id
    except sqlite3.IntegrityError as exc:
        conn.rollback()
        raise ValidationError(f"Database rejected the lender profile: {exc}")


def register_borrower(conn, data, auto_activate=False):
    """Validate and store a borrower.  Returns the new borrower_id."""
    errors = []
    f = lambda key, label: _number(data, key, label, float, errors)       # noqa: E731
    borrower = Borrower(
        _text(data, "full_name"), _text(data, "email"),
        f("monthly_income", "Monthly income"), f("existing_obligations", "Existing obligations"),
        _number(data, "credit_score", "Credit score", int, errors),
        f("credit_history_years", "Credit history (years)"))
    password_hash = _check_password(data, errors)
    if errors:
        raise ValidationError(errors)
    errors = borrower.validate()
    if borrower.email and _email_taken(conn, borrower.email):
        errors.append("This email address is already registered.")
    if errors:
        raise ValidationError(errors)

    status = "active" if auto_activate else "pending"
    try:
        user_id = _insert_user(conn, borrower, "borrower", password_hash)
        cur = conn.execute(
            "INSERT INTO borrowers(user_id, monthly_income, existing_obligations, credit_score,"
            " credit_history_years, profile_status) VALUES (?, ?, ?, ?, ?, ?)",
            (user_id, borrower.monthly_income, borrower.existing_obligations, borrower.credit_score,
             borrower.credit_history_years, status))
        borrower_id = cur.lastrowid
        audit.log(conn, "borrower", borrower_id, "REGISTERED", None, status, actor=borrower.email)
        conn.commit()
        return borrower_id
    except sqlite3.IntegrityError as exc:
        conn.rollback()
        raise ValidationError(f"Database rejected the borrower profile: {exc}")


def create_loan_request(conn, borrower_id, data):
    """Validate and store a loan request for an ACTIVE borrower.  Returns request_id."""
    borrower = get_borrower(conn, borrower_id)
    if borrower is None:
        raise ValidationError("Borrower does not exist.")
    if not borrower.is_matchable():
        raise ValidationError(f"Borrower profile must be active before requesting a loan "
                              f"(current status: {borrower.profile_status}).")
    errors = []
    request = LoanRequest(borrower_id,
                          _number(data, "amount", "Loan amount", float, errors),
                          _number(data, "term_months", "Repayment term", int, errors),
                          _text(data, "purpose"))
    if errors:
        raise ValidationError(errors)
    errors = request.validate()
    duplicate = conn.execute(
        "SELECT request_id FROM loan_requests WHERE borrower_id = ? AND amount = ? AND term_months = ?"
        " AND status IN ('pending', 'matched')", (borrower_id, request.amount, request.term_months)).fetchone()
    if duplicate:
        errors.append(f"Duplicate request: request #{duplicate['request_id']} with the same amount and "
                      f"term is already open.")
    if errors:
        raise ValidationError(errors)
    try:
        cur = conn.execute(
            "INSERT INTO loan_requests(borrower_id, amount, term_months, purpose) VALUES (?, ?, ?, ?)",
            (borrower_id, request.amount, request.term_months, request.purpose))
        request_id = cur.lastrowid
        audit.log(conn, "loan_request", request_id, "CREATED", None, "pending", actor=borrower.email)
        conn.commit()
        return request_id
    except sqlite3.IntegrityError as exc:
        conn.rollback()
        raise ValidationError(f"Database rejected the loan request: {exc}")


def cancel_request(conn, request_id, actor="user"):
    row = conn.execute("SELECT status FROM loan_requests WHERE request_id = ?", (request_id,)).fetchone()
    if row is None:
        raise ValidationError("Loan request does not exist.")
    if row["status"] not in ("pending", "matched"):
        raise ValidationError(f"A request in status '{row['status']}' cannot be cancelled.")
    conn.execute("UPDATE loan_requests SET status = 'cancelled' WHERE request_id = ?", (request_id,))
    audit.log(conn, "loan_request", request_id, "STATUS_CHANGE", row["status"], "cancelled", actor)
    conn.commit()


# ---------------------------------------------------------------------------
# profile-status tracking
# ---------------------------------------------------------------------------
```

```
def set_profile_status(conn, role, profile_id, new_status, actor="admin"):
    """Move a lender/borrower profile along the allowed status transitions."""
    if role not in ("lender", "borrower"):
        raise ValidationError("Role must be 'lender' or 'borrower'.")
    table, key = ("lenders", "lender_id") if role == "lender" else ("borrowers", "borrower_id")
    row = conn.execute(f"SELECT profile_status FROM {table} WHERE {key} = ?", (profile_id,)).fetchone()
    if row is None:
        raise ValidationError(f"{role.capitalize()} #{profile_id} does not exist.")
    old = row["profile_status"]
    if new_status not in STATUS_TRANSITIONS.get(old, set()):
        raise ValidationError(f"Cannot change {role} profile from '{old}' to '{new_status}'.")
    conn.execute(f"UPDATE {table} SET profile_status = ? WHERE {key} = ?", (new_status, profile_id))
    audit.log(conn, role, profile_id, "PROFILE_STATUS_CHANGE", old, new_status, actor)
    conn.commit()


# ---------------------------------------------------------------------------
# queries
# ---------------------------------------------------------------------------
def get_lender(conn, lender_id):
    row = conn.execute(_LENDER_SQL + " WHERE l.lender_id = ?", (lender_id,)).fetchone()
    return Lender.from_row(row) if row else None


def get_borrower(conn, borrower_id):
    row = conn.execute(_BORROWER_SQL + " WHERE b.borrower_id = ?", (borrower_id,)).fetchone()
    return Borrower.from_row(row) if row else None


def get_request(conn, request_id):
    row = conn.execute("SELECT * FROM loan_requests WHERE request_id = ?", (request_id,)).fetchone()
    return LoanRequest.from_row(row) if row else None


def list_lenders(conn, profile_status=None):
    sql, params = _LENDER_SQL, ()
    if profile_status:
        sql, params = sql + " WHERE l.profile_status = ?", (profile_status,)
    return [Lender.from_row(r) for r in conn.execute(sql + " ORDER BY l.lender_id", params)]


def list_borrowers(conn, profile_status=None):
    sql, params = _BORROWER_SQL, ()
    if profile_status:
        sql, params = sql + " WHERE b.profile_status = ?", (profile_status,)
    return [Borrower.from_row(r) for r in conn.execute(sql + " ORDER BY b.borrower_id", params)]


def list_requests(conn, status=None):
    """Loan requests joined with the borrower's name (plain dicts for tables)."""
    sql = ("SELECT r.request_id, r.borrower_id, u.full_name AS borrower, r.amount, r.term_months,"
           " r.purpose, r.status, r.created_at FROM loan_requests r"
           " JOIN borrowers b ON b.borrower_id = r.borrower_id JOIN users u ON u.user_id = b.user_id")
    params = ()
    if status:
        sql, params = sql + " WHERE r.status = ?", (status,)
    return [dict(r) for r in conn.execute(sql + " ORDER BY r.request_id", params)]
```

[NEW] matching_engine.py

```
"""Week 2 skeleton of the MatchingEngine class.

Week 2 only needs the engine to LOAD its inputs (active lenders and pending
requests).  Week 3 adds scoring and Week 4 replaces this file with the full
graph-based engine, so nothing written here is thrown away.
"""
import profile_service


class MatchingEngine:
    def __init__(self, conn, allow_partial=False):
        self.conn = conn
        self.allow_partial = allow_partial

    def load_active_lenders(self):
        return profile_service.list_lenders(self.conn, profile_status="active")

    def load_pending_requests(self):
        return profile_service.list_requests(self.conn, status="pending")

    def run(self):
        raise NotImplementedError("Matching is implemented in Week 4.")
```

[NEW] routes/__init__.py

```
(empty file - just create it with this name)
```

[NEW] routes/profiles.py

```
"""Week 2 routes - lender / borrower registration, loan requests, profile status."""
from flask import Blueprint, flash, redirect, render_template, request, url_for

import profile_service as ps
from models import ValidationError
from utils import db_conn

bp = Blueprint("profiles", __name__)

RISK_OPTIONS = [("low", "Low"), ("medium", "Medium"), ("high", "High")]


def _flash_errors(exc):
    for message in exc.errors:
        flash(message, "danger")


def _status_actions(role, pid, status):
    """Buttons shown for each profile row depending on its current status."""
    base = url_for("profiles.change_status", role=role, profile_id=pid)
    options = {"pending": [("Approve", "active", "success"), ("Reject", "rejected", "danger")],
               "active": [("Deactivate", "inactive", "warning")],
               "inactive": [("Activate", "active", "success")],
               "rejected": [("Re-open", "pending", "secondary")]}
    return [{"label": label, "url": f"{base}?to={to}", "method": "post", "style": style}
            for label, to, style in options.get(status, [])]


# ---------------------------------------------------------------------------
@bp.route("/lenders/register", methods=["GET", "POST"])
def register_lender():
    fields = [
        {"name": "full_name", "label": "Full name"},
        {"name": "email", "label": "Email", "type": "email"},
        {"name": "password", "label": "Password (min 8 characters)", "type": "password"},
        {"name": "risk_appetite", "label": "Risk appetite", "type": "select", "options": RISK_OPTIONS},
        {"name": "available_amount", "label": "Available investment amount", "type": "number", "step": "any"},
        {"name": "min_loan_amount", "label": "Minimum loan amount you will fund", "type": "number", "step": "any"},
        {"name": "max_loan_amount", "label": "Maximum loan amount you will fund", "type": "number", "step": "any"},
        {"name": "min_duration_months", "label": "Preferred minimum duration (months)", "type": "number"},
        {"name": "max_duration_months", "label": "Preferred maximum duration (months)", "type": "number"},
        {"name": "min_expected_return", "label": "Minimum expected return (% per year)", "type": "number", "step": "any"},
        {"name": "max_expected_return", "label": "Maximum expected return (% per year)", "type": "number", "step": "any"},
    ]
    if request.method == "POST":
        with db_conn() as conn:
            try:
                lender_id = ps.register_lender(conn, request.form)
                flash(f"Lender #{lender_id} registered. Status: pending approval.", "success")
                return redirect(url_for("profiles.lenders"))
            except ValidationError as exc:
                _flash_errors(exc)
    return render_template("form.html", title="Lender registration", fields=fields,
                           values=request.form, submit_label="Register lender")


@bp.route("/borrowers/register", methods=["GET", "POST"])
def register_borrower():
    fields = [
        {"name": "full_name", "label": "Full name"},
        {"name": "email", "label": "Email", "type": "email"},
        {"name": "password", "label": "Password (min 8 characters)", "type": "password"},
        {"name": "monthly_income", "label": "Simulated monthly income", "type": "number", "step": "any"},
        {"name": "existing_obligations", "label": "Existing monthly obligations (EMIs)", "type": "number", "step": "any"},
        {"name": "credit_score", "label": "Simulated credit score (300-900)", "type": "number"},
        {"name": "credit_history_years", "label": "Credit history (years)", "type": "number", "step": "any"},
    ]
    if request.method == "POST":
        with db_conn() as conn:
            try:
                borrower_id = ps.register_borrower(conn, request.form)
                flash(f"Borrower #{borrower_id} registered. Status: pending approval.", "success")
                return redirect(url_for("profiles.borrowers"))
            except ValidationError as exc:
                _flash_errors(exc)
    return render_template("form.html", title="Borrower registration", fields=fields,
                           values=request.form, submit_label="Register borrower")


@bp.route("/requests/new", methods=["GET", "POST"])
def new_request():
    with db_conn() as conn:
        active = [(b.borrower_id, f"#{b.borrower_id} {b.full_name}")
                  for b in ps.list_borrowers(conn, profile_status="active")]
        fields = [
            {"name": "borrower_id", "label": "Borrower", "type": "select", "options": active},
            {"name": "amount", "label": "Requested amount", "type": "number", "step": "any"},
            {"name": "term_months", "label": "Repayment term (months)", "type": "number"},
            {"name": "purpose", "label": "Purpose (optional)"},
        ]
        if request.method == "POST":
            try:
                borrower_id = int(request.form.get("borrower_id", ""))
                request_id = ps.create_loan_request(conn, borrower_id, request.form)
                flash(f"Loan request #{request_id} created.", "success")
                return redirect(url_for("profiles.requests"))
            except ValueError:
                flash("Please choose a borrower (only active borrowers can request loans).", "danger")
```

```
            except ValidationError as exc:
                _flash_errors(exc)
    return render_template("form.html", title="New loan request", fields=fields,
                           values=request.form, submit_label="Submit request")


# ---------------------------------------------------------------------------
@bp.route("/lenders")
def lenders():
    with db_conn() as conn:
        rows = []
        for lender in ps.list_lenders(conn):
            row = lender.to_dict()
            row["_actions"] = _status_actions("lender", lender.lender_id, lender.profile_status)
            rows.append(row)
    columns = [("lender_id", "ID"), ("full_name", "Name"), ("email", "Email"), ("risk_appetite", "Risk"),
               ("available_amount", "Available"), ("min_loan_amount", "Min loan"), ("max_loan_amount", "Max loan"),
               ("min_duration_months", "Min m"), ("max_duration_months", "Max m"),
               ("min_expected_return", "Min %"), ("max_expected_return", "Max %"), ("profile_status", "Status")]
    return render_template("table.html", title="Lenders", columns=columns, rows=rows,
                           top_actions=[{"label": "+ Register lender", "url": url_for("profiles.register_lender"),
                                         "method": "get"}])


@bp.route("/borrowers")
def borrowers():
    with db_conn() as conn:
        rows = []
        for borrower in ps.list_borrowers(conn):
            row = borrower.to_dict()
            row["_actions"] = _status_actions("borrower", borrower.borrower_id, borrower.profile_status)
            rows.append(row)
    columns = [("borrower_id", "ID"), ("full_name", "Name"), ("email", "Email"), ("monthly_income", "Income"),
               ("existing_obligations", "Obligations"), ("credit_score", "Credit"),
               ("credit_history_years", "History (y)"), ("profile_status", "Status")]
    return render_template("table.html", title="Borrowers", columns=columns, rows=rows,
                           top_actions=[{"label": "+ Register borrower", "url": url_for("profiles.register_borrower"),
                                         "method": "get"}])


@bp.route("/requests")
def requests():
    with db_conn() as conn:
        rows = ps.list_requests(conn)
    for row in rows:
        if row["status"] in ("pending", "matched"):
            row["_actions"] = [{"label": "Cancel", "method": "post", "style": "outline-danger",
                                "url": url_for("profiles.cancel", request_id=row["request_id"])}]
    columns = [("request_id", "ID"), ("borrower", "Borrower"), ("amount", "Amount"), ("term_months", "Term (m)"),
               ("purpose", "Purpose"), ("status", "Status"), ("created_at", "Created")]
    return render_template("table.html", title="Loan requests", columns=columns, rows=rows,
                           top_actions=[{"label": "+ New request", "url": url_for("profiles.new_request"),
                                         "method": "get"}])


@bp.route("/profiles/<role>/<int:profile_id>/status", methods=["POST"])
def change_status(role, profile_id):
    new_status = request.args.get("to", "")
    with db_conn() as conn:
        try:
            ps.set_profile_status(conn, role, profile_id, new_status)
            flash(f"{role.capitalize()} #{profile_id} is now '{new_status}'.", "success")
        except ValidationError as exc:
            _flash_errors(exc)
    return redirect(url_for("profiles.lenders" if role == "lender" else "profiles.borrowers"))


@bp.route("/requests/<int:request_id>/cancel", methods=["POST"])
def cancel(request_id):
    with db_conn() as conn:
        try:
            ps.cancel_request(conn, request_id)
            flash(f"Request #{request_id} cancelled.", "success")
        except ValidationError as exc:
            _flash_errors(exc)
    return redirect(url_for("profiles.requests"))
```

[NEW] demo_data.py

```
"""Hand-made demo dataset (simulated people) used by the weekly demo scripts.

Loaded through the SAME validated services the web forms use, so the demo
also proves that registration works.  Run  python demo_data.py  to load the
data into the real application database (data/p2p_lending.db).
"""
import os

import config
import database
import profile_service as ps

PASSWORD = "password123"

LENDERS = [
    # name, email, appetite, available, min_loan, max_loan, min_m, max_m, min_ret, max_ret
    ("Conservative Carla", "carla@p2p.test", "low", 300000, 10000, 150000, 6, 36, 7, 11),
    ("Balanced Bala", "bala@p2p.test", "medium", 250000, 5000, 200000, 6, 48, 9, 15),
    ("Aggressive Arjun", "arjun@p2p.test", "high", 250000, 5000, 300000, 3, 60, 12, 22),
    ("Small Sita", "sita@p2p.test", "medium", 20000, 2000, 30000, 3, 24, 9, 16),
    ("Cohort Chitra", "chitra@p2p.test", "high", 100000, 50000, 300000, 12, 60, 12, 22),
]

BORROWERS = [
    # name, email, income, obligations, credit, history_years
    ("Ravi Kumar", "ravi@p2p.test", 60000, 8000, 780, 6),
    ("Meena Iyer", "meena@p2p.test", 25000, 9000, 540, 1.5),
    ("Sunil Verma", "sunil@p2p.test", 45000, 12000, 650, 3),
    ("Anita Das", "anita@p2p.test", 30000, 18000, 600, 2),
    ("Kiran Rao", "kiran@p2p.test", 90000, 5000, 820, 10),
    ("Pooja Nair", "pooja@p2p.test", 35000, 6000, 700, 4),
]

REQUESTS = [
    # borrower index (1-based), amount, term, purpose
    (1, 50000, 12, "Education"),
    (2, 20000, 6, "Medical"),
    (3, 80000, 24, "Home renovation"),
    (4, 40000, 12, "Business"),
    (5, 15000, 12, "Vehicle repair"),
    (5, 120000, 36, "Home loan top-up"),
    (6, 280000, 48, "Start-up capital"),
]


def load_demo_data(conn):
    """Insert the demo dataset; returns (lender_ids, borrower_ids, request_ids)."""
    lender_ids = []
    for name, email, appetite, avail, lo, hi, mn, mx, rmin, rmax in LENDERS:
        lender_ids.append(ps.register_lender(conn, {
            "full_name": name, "email": email, "password": PASSWORD, "risk_appetite": appetite,
            "available_amount": avail, "min_loan_amount": lo, "max_loan_amount": hi,
            "min_duration_months": mn, "max_duration_months": mx,
            "min_expected_return": rmin, "max_expected_return": rmax}, auto_activate=True))
    borrower_ids = []
    for name, email, income, obligations, credit, years in BORROWERS:
        borrower_ids.append(ps.register_borrower(conn, {
            "full_name": name, "email": email, "password": PASSWORD, "monthly_income": income,
            "existing_obligations": obligations, "credit_score": credit,
            "credit_history_years": years}, auto_activate=True))
    request_ids = []
    for index, amount, term, purpose in REQUESTS:
        request_ids.append(ps.create_loan_request(conn, borrower_ids[index - 1], {
            "amount": amount, "term_months": term, "purpose": purpose}))
    return lender_ids, borrower_ids, request_ids


def fresh_demo_db(path=None):
    """Create a brand-new demo database file, load the demo data, return a connection."""
    path = path or os.path.join(config.BASE_DIR, "data", "demo.db")
    if os.path.exists(path):
        os.remove(path)
    database.init_db(path)
    conn = database.get_connection(path)
    load_demo_data(conn)
    return conn


if __name__ == "__main__":
    database.init_db()
    connection = database.get_connection()
    if connection.execute("SELECT COUNT(*) FROM users").fetchone()[0]:
        print("The application database already contains users - nothing loaded.")
        print("Delete data/p2p_lending.db first if you want a clean start.")
    else:
        load_demo_data(connection)
        print("Demo data loaded into", config.DATABASE_PATH)
    connection.close()
```

[NEW] demo_week2.py

```
"""Week 2 demo: profiles, validation and status tracking (uses data/demo.db)."""
import demo_data
import profile_service as ps
from models import ValidationError


def show(title):
    print("\n" + "=" * 70 + f"\n{title}\n" + "=" * 70)


conn = demo_data.fresh_demo_db()

show("1. Registered lenders")
for l in ps.list_lenders(conn):
    print(f"#{l.lender_id} {l.full_name:<20} appetite={l.risk_appetite:<6} available={l.available_amount:>10,.0f} "
          f"loans {l.min_loan_amount:,.0f}-{l.max_loan_amount:,.0f}  {l.min_duration_months}-{l.max_duration_months}m "
          f" return {l.min_expected_return:g}-{l.max_expected_return:g}%  [{l.profile_status}]")

show("2. Registered borrowers")
for b in ps.list_borrowers(conn):
    print(f"#{b.borrower_id} {b.full_name:<14} income={b.monthly_income:>8,.0f} obligations={b.existing_obligations:>7,.0f} "
          f"credit={b.credit_score} history={b.credit_history_years}y  DTI={b.debt_to_income():.0%}  [{b.profile_status}]")

show("3. Loan requests")
for r in ps.list_requests(conn):
    print(f"#{r['request_id']} {r['borrower']:<14} amount={r['amount']:>9,.0f} term={r['term_months']}m "
          f"purpose={r['purpose']:<16} [{r['status']}]")

show("4. Validation rejects wrong data")
bad_cases = {
    "missing name + invalid email": {"full_name": "", "email": "not-an-email", "password": "password123",
        "risk_appetite": "low", "available_amount": 1000, "min_loan_amount": 1000, "max_loan_amount": 5000,
        "min_duration_months": 3, "max_duration_months": 12, "min_expected_return": 5, "max_expected_return": 9},
    "unknown risk appetite": {"full_name": "Test Lender", "email": "t1@p2p.test", "password": "password123",
        "risk_appetite": "extreme", "available_amount": 1000, "min_loan_amount": 1000, "max_loan_amount": 5000,
        "min_duration_months": 3, "max_duration_months": 12, "min_expected_return": 5, "max_expected_return": 9},
    "min duration above max": {"full_name": "Test Lender", "email": "t2@p2p.test", "password": "password123",
        "risk_appetite": "low", "available_amount": 1000, "min_loan_amount": 1000, "max_loan_amount": 5000,
        "min_duration_months": 24, "max_duration_months": 12, "min_expected_return": 5, "max_expected_return": 9},
    "text in a number field": {"full_name": "Test Lender", "email": "t3@p2p.test", "password": "password123",
        "risk_appetite": "low", "available_amount": "lots", "min_loan_amount": 1000, "max_loan_amount": 5000,
        "min_duration_months": 3, "max_duration_months": 12, "min_expected_return": 5, "max_expected_return": 9},
    "duplicate email": {"full_name": "Another Carla", "email": "carla@p2p.test", "password": "password123",
        "risk_appetite": "low", "available_amount": 1000, "min_loan_amount": 1000, "max_loan_amount": 5000,
        "min_duration_months": 3, "max_duration_months": 12, "min_expected_return": 5, "max_expected_return": 9},
}
for label, payload in bad_cases.items():
    try:
        ps.register_lender(conn, payload)
        print(f"- {label}: ACCEPTED (unexpected!)")
    except ValidationError as exc:
        print(f"- {label}: rejected -> {exc.errors[0]}")
try:
    ps.create_loan_request(conn, 1, {"amount": 50000, "term_months": 12})
except ValidationError as exc:
    print(f"- duplicate loan request: rejected -> {exc.errors[0]}")

show("5. Profile-status tracking (pending -> active -> inactive)")
new_id = ps.register_lender(conn, {"full_name": "Pending Pia", "email": "pia@p2p.test", "password": "password123",
    "risk_appetite": "medium", "available_amount": 50000, "min_loan_amount": 5000, "max_loan_amount": 40000,
    "min_duration_months": 6, "max_duration_months": 24, "min_expected_return": 8, "max_expected_return": 14})
print("registered lender status :", ps.get_lender(conn, new_id).profile_status)
ps.set_profile_status(conn, "lender", new_id, "active")
print("after approval           :", ps.get_lender(conn, new_id).profile_status)
ps.set_profile_status(conn, "lender", new_id, "inactive")
print("after deactivation       :", ps.get_lender(conn, new_id).profile_status)
try:
    ps.set_profile_status(conn, "lender", new_id, "rejected")
except ValidationError as exc:
    print("illegal move inactive->rejected blocked:", exc.errors[0])

show("6. Audit trail (last 5 entries)")
for row in conn.execute("SELECT entity_type, entity_id, action, old_value, new_value FROM audit_logs "
                        "ORDER BY log_id DESC LIMIT 5"):
    print(tuple(row))
conn.close()
```

[NEW] tests/__init__.py

```
(empty file - just create it with this name)
```

[NEW] *tests/test_week2.py*

```
"""Week 2 tests: models, validation, profile status tracking, forms.
Run:  python -m unittest tests.test_week2 -v     (or  python -m pytest tests/test_week2.py)
"""
import os
import sys
import tempfile
import unittest

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
import database
import profile_service as ps
from models import Borrower, Lender, LoanRequest, ValidationError, calculate_emi

LENDER = {"full_name": "Test Lender", "email": "tl@x.com", "password": "password123", "risk_appetite": "medium",
          "available_amount": 100000, "min_loan_amount": 5000, "max_loan_amount": 50000,
          "min_duration_months": 6, "max_duration_months": 24, "min_expected_return": 8, "max_expected_return": 15}
BORROWER = {"full_name": "Test Borrower", "email": "tb@x.com", "password": "password123", "monthly_income": 40000,
            "existing_obligations": 5000, "credit_score": 700, "credit_history_years": 4}


class Week2Tests(unittest.TestCase):
    def setUp(self):
        self.path = os.path.join(tempfile.mkdtemp(), "t.db")
        database.init_db(self.path)
        self.conn = database.get_connection(self.path)

    def tearDown(self):
        self.conn.close()

    def expect_error(self, func, *args, contains=""):
        with self.assertRaises(ValidationError) as ctx:
            func(*args)
        self.assertIn(contains.lower(), " ".join(ctx.exception.errors).lower())

    # ---- models ------------------------------------------------------------
    def test_emi_formula(self):
        self.assertAlmostEqual(calculate_emi(100000, 12, 12), 8884.88, places=2)
        self.assertEqual(calculate_emi(1200, 0, 12), 100.0)

    def test_lender_behaviour(self):
        lender = Lender("Ann Lee", "a@x.com", "medium", 1000, 100, 900, 6, 12, 8, 12, profile_status="active")
        self.assertTrue(lender.accepts_risk("low") and lender.accepts_risk("medium"))
        self.assertFalse(lender.accepts_risk("high"))
        self.assertTrue(lender.accepts_duration(12) and not lender.accepts_duration(13))
        self.assertTrue(lender.accepts_rate(8) and not lender.accepts_rate(12.5))
        self.assertTrue(lender.is_matchable())

    def test_borrower_dti_and_request_instalment(self):
        b = Borrower("Bo Ray", "b@x.com", 10000, 2500, 700, 3)
        self.assertEqual(b.debt_to_income(), 0.25)
        self.assertEqual(LoanRequest(1, 100000, 12).monthly_instalment(12), 8884.88)

    # ---- registration + validation -----------------------------------------
    def test_register_lender_starts_pending(self):
        lender_id = ps.register_lender(self.conn, LENDER)
        self.assertEqual(ps.get_lender(self.conn, lender_id).profile_status, "pending")
        self.assertEqual(self.conn.execute("SELECT role FROM users").fetchone()["role"], "lender")

    def test_password_is_hashed(self):
        ps.register_lender(self.conn, LENDER)
        stored = self.conn.execute("SELECT password_hash FROM users").fetchone()[0]
        self.assertNotIn("password123", stored)

    def test_missing_fields(self):
        self.expect_error(ps.register_lender, self.conn, {}, contains="required")
        self.expect_error(ps.register_borrower, self.conn, {}, contains="required")

    def test_invalid_values(self):
        self.expect_error(ps.register_lender, self.conn, dict(LENDER, risk_appetite="wild"), contains="risk appetite")
        self.expect_error(ps.register_lender, self.conn, dict(LENDER, email="bad"), contains="valid email")
        self.expect_error(ps.register_lender, self.conn, dict(LENDER, available_amount="-5"), contains="greater than 0")
        self.expect_error(ps.register_lender, self.conn, dict(LENDER, min_duration_months="30"), contains="duration")
        self.expect_error(ps.register_lender, self.conn, dict(LENDER, available_amount="nan"), contains="finite")
        self.expect_error(ps.register_lender, self.conn, dict(LENDER, password="short"), contains="password")
        self.expect_error(ps.register_borrower, self.conn, dict(BORROWER, credit_score="950"), contains="credit score")
        self.expect_error(ps.register_borrower, self.conn, dict(BORROWER, monthly_income="0"), contains="income")
        self.expect_error(ps.register_borrower, self.conn, dict(BORROWER, credit_score="7.5"), contains="whole number")
        self.assertEqual(self.conn.execute("SELECT COUNT(*) FROM users").fetchone()[0], 0)

    def test_duplicate_email(self):
        ps.register_lender(self.conn, LENDER)
        self.expect_error(ps.register_borrower, self.conn, dict(BORROWER, email="TL@x.com"), contains="already")

    # ---- loan requests -----------------------------------------------------
    def test_request_needs_active_borrower(self):
        bid = ps.register_borrower(self.conn, BORROWER)
        self.expect_error(ps.create_loan_request, self.conn, bid, {"amount": 5000, "term_months": 12}, contains="active")
        ps.set_profile_status(self.conn, "borrower", bid, "active")
        rid = ps.create_loan_request(self.conn, bid, {"amount": 5000, "term_months": 12})
        self.assertEqual(ps.get_request(self.conn, rid).status, "pending")

    def test_duplicate_and_invalid_requests(self):
        bid = ps.register_borrower(self.conn, BORROWER, auto_activate=True)
        ps.create_loan_request(self.conn, bid, {"amount": 5000, "term_months": 12})
        self.expect_error(ps.create_loan_request, self.conn, bid, {"amount": 5000, "term_months": 12}, contains="duplicate")
        self.expect_error(ps.create_loan_request, self.conn, bid, {"amount": 10, "term_months": 12}, contains="between")
        self.expect_error(ps.create_loan_request, self.conn, bid, {"amount": 5000, "term_months": 500}, contains="term")
        self.expect_error(ps.create_loan_request, self.conn, bid, {"amount": "", "term_months": 12}, contains="required")
```

```
        self.expect_error(ps.create_loan_request, self.conn, 999, {"amount": 5000, "term_months": 12}, contains="exist")

    def test_cancel_request_allows_same_request_again(self):
        bid = ps.register_borrower(self.conn, BORROWER, auto_activate=True)
        rid = ps.create_loan_request(self.conn, bid, {"amount": 5000, "term_months": 12})
        ps.cancel_request(self.conn, rid)
        ps.create_loan_request(self.conn, bid, {"amount": 5000, "term_months": 12})

    # ---- profile-status tracking ------------------------------------------
    def test_status_transitions_and_audit(self):
        lid = ps.register_lender(self.conn, LENDER)
        ps.set_profile_status(self.conn, "lender", lid, "active")
        ps.set_profile_status(self.conn, "lender", lid, "inactive")
        self.expect_error(ps.set_profile_status, self.conn, "lender", lid, "rejected", contains="cannot change")
        self.expect_error(ps.set_profile_status, self.conn, "lender", 99, "active", contains="does not exist")
        actions = [r[0] for r in self.conn.execute("SELECT action FROM audit_logs WHERE entity_type='lender'")]
        self.assertEqual(actions, ["REGISTERED", "PROFILE_STATUS_CHANGE", "PROFILE_STATUS_CHANGE"])

    def test_engine_loads_only_active_lenders(self):
        from matching_engine import MatchingEngine
        ps.register_lender(self.conn, LENDER, auto_activate=True)
        ps.register_lender(self.conn, dict(LENDER, email="o@x.com"))
        self.assertEqual(len(MatchingEngine(self.conn).load_active_lenders()), 1)

    # ---- web forms --------------------------------------------------------
    def test_forms_in_browser(self):
        from app import create_app
        client = create_app(self.path).test_client()
        for url in ("/", "/lenders", "/borrowers", "/requests", "/lenders/register", "/borrowers/register", "/requests/new"):
            self.assertEqual(client.get(url).status_code, 200, url)
        bad = client.post("/lenders/register", data=dict(LENDER, email="bad"))
        self.assertEqual(bad.status_code, 200)
        self.assertIn(b"valid email", bad.data)
        ok = client.post("/lenders/register", data=LENDER, follow_redirects=True)
        self.assertIn(b"registered", ok.data)
        client.post("/borrowers/register", data=BORROWER)
        approve = client.post("/profiles/borrower/1/status?to=active", follow_redirects=True)
        self.assertIn(b"is now", approve.data)
        made = client.post("/requests/new", data={"borrower_id": "1", "amount": "9000", "term_months": "12"},
                           follow_redirects=True)
        self.assertIn(b"created", made.data)


if __name__ == "__main__":
    unittest.main(verbosity=2)
```

Screenshots to attach to your submission

- Lender registration form and borrower registration form.
- A validation error (for example invalid e-mail) shown in red.
- /lenders page with a pending profile and the Approve button, then the same page after approval.
- Terminal output of demo_week2.py and of the 14 passing tests.